

FPT UNIVERSITY

GRADUATION THESIS

Using Lightweight Graph Convolutional Neural Network in Recommender System

Quang Dung Nguyen
Quoc Lap Dinh
Ba Hoang Nam Nguyen
Supervisor: PhD. Van Hieu Bui

Bachelor of Artificial Intelligence
FPT University
December 19, 2024

Abstract

Traditional graph neural network (GNN)-based recommendation systems encounter significant challenges in managing high computational costs and scalability when modeling complex, heterogeneous user-query interactions. These models frequently fall short in capturing the diversity of relationship types and multi-level contextual information, resulting in limited performance gains. To overcome these limitations, we present the Heterogeneous Light Graph Convolutional Network (He-LiGCN), a streamlined framework designed to enhance both accuracy and scalability. Leveraging multi-level graph coarsening and sophisticated contextual embeddings, He-LiGCN efficiently models intricate user-item dependencies with substantially reduced computational demands. Experimental results demonstrate a 6.5% improvement in Recall@10 and an 8.9% increase in NDCG@10 over baseline models, along with a reduction in training time by up to 20%, underscoring its effectiveness and efficiency in large-scale recommendation scenarios. By advancing the efficiency and depth of graph-based recommendations, He-LiGCN represents a practical and scalable solution for the next generation of personalized recommendation systems.

Keywords: Convolutional neural network · Heterogeneous graph analysis · Recommendation system · Scalability

Acknowledgments

Deep gratitude is extended to the many individuals whose support and guidance have been instrumental throughout the academic journey leading to this graduation thesis. Foremost recognition is given to Dr. Van Hieu Bui for his invaluable guidance, consistent feedback, and constructive insights, which were pivotal in shaping the direction and success of this work.

Equal appreciation is conveyed to groupmates, Lap and Nam, for their dedication and collaborative efforts in completing this thesis. Acknowledgment is also made of the unwavering support from friends, who provided balance and encouragement during challenging moments.

Lastly, profound thanks are owed to parents, whose love and encouragement have been an enduring source of strength.

Contents

List of Figures v

List of Tables vi

N Notation & Conventions vii

1 Introduction 1

2 Related Work 3

3 Methodology 9

3.1 Preprocessing: Constructing Efficient Heterogeneous Graphs . . . 10

3.1.1. Node Similarity with Jaccard Index 10 • 3.1.2. Locality Sensitive Hashing: More Efficient Node Grouping 11

3.2 Supernode Embedding . . . 12

3.2.1. Generating Initial Embeddings with General Attributed Multiplex Heterogeneous Network Embedding 13 • 3.2.2. Loss Function 15

3.3 Refinement . . . 16

3.3.1. Enhancing Representations with Heterogeneous Graph Convolutional Networks 17 • 3.3.2. Loss Function 19

3.4 Prediction and Recommendation . . . 19

3.4.1. Efficient Information Aggregation with Light Graph Convolution 20 • 3.4.2. Resolving Over-smoothing Problem with Approximate Personalized Propagation of Neural Predictions 21 • 3.4.3. Matrix Representation 22 • 3.4.4. Loss Function 23

4 Experiments and Results 24

4.1 Experiments . . . 24

4.1.1. Dataset Preparation 24 • 4.1.2. Training Parameters 24 • 4.1.3. Evaluation Metrics 25

4.2 Results . . . 26

4.2.1. Comparative Analysis of Recommendation Models 26 • 4.2.2. Scalability and Efficiency Across Graph Layers 27 • 4.2.3. Ablation and Effectiveness Studies 27

5 Conclusion 29

5.1 Key Contributions . . . 29

5.2 Limitations . . . 29

5.3 Future Work . . . 30

References 31

List of Figures

3.1 The He-LiGCN workflow. 9
3.2 Initialized heterogeneous graph process. 10
3.3 Coarsening process. 11
3.6 LSH process. 12
3.10 The components of the GATNE model. 13
3.11 Random walk implementation. 14
3.17 Example of negative sample (random walk 1) and positive samples (random walk 2). 16
3.19 The process of embedding generation for two layers, G_m and G_{m-1}, using hierarchical projection and representation learning. 17
3.20 The architecture of HGCN. 18
3.25 The architecture of LGC model. 20
3.31 Over-smoothing problem. 21
3.33 Adjacency matrix representation. 22

4.2 The training performance of NGCF, LightGCN, and He-LiGCN models over 200 epochs. 26

5.1 Sparse matrix representation. 30

List of Tables

4.1 Training Parameters for He-LiGCN, NGCF, and LightGCN 24
4.3 Performance comparison at different layer depths. 27
4.4 Ablation study results for He-LiGCN, showing Recall@10 and NDCG@10 performance with removed components (1: graph coarsening; 2: GATNE and HGCN; 3: LGC). 27
4.5 Average computational cost per epoch for He-LiGCN, NGCF, and LightGCN. 28

Notation & Conventions

Symbol/Term	Definition/Description
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$	A heterogeneous graph, where $\mathcal{V}$ is the set of nodes, $\mathcal{E}$ is the set of edges, and $\mathcal{A}$ represents attributes.
u, v	Nodes in the graph, representing users or items.
$\mathcal{N}_u$	Neighborhood of node u, i.e., the set of nodes directly connected to u.
$J(u, v)$	Jaccard similarity between nodes u and v.
$\mathcal{G}_0, \mathcal{G}_1, \ldots, \mathcal{G}_m$	Sequence of coarsened graphs, where $\mathcal{G}_0$ is the original graph, and $\mathcal{G}_m$ is the coarsest graph.
$\boldsymbol{E}^{(k)}$	Node embedding matrix at layer k.
$\boldsymbol{A}$	Adjacency matrix of the graph.
$\hat{\boldsymbol{A}}$	Normalized adjacency matrix of the graph.
$\boldsymbol{e}_u, \boldsymbol{e}_i$	Final embeddings for user u and item i, respectively.
$\hat{y}_{ui}$	Predicted score for interaction between user u and item i, computed as $\hat{y}_{ui} = \boldsymbol{e}_u^\top \boldsymbol{e}_i$.

1 Introduction

Artificial intelligence has brought about breakthrough achievements across various fields, with recommendation systems emerging as one of its most impactful applications [1]. In the era of exponential digital data growth, these systems have become indispensable tools for enhancing user experience on online platforms. From e-commerce giants like Amazon and Shopee to entertainment services such as Netflix, Spotify, and YouTube, recommendation systems play a crucial role in connecting users with relevant content while generating significant economic value [2].

Consider a specific scenario on an online movie streaming platform. User A, who loves science fiction movies, frequently watches films like "Interstellar", "The Martian", and "Blade Runner". Meanwhile, User B prefers action movies, often searching for titles such as "John Wick", "Die Hard", or "Mad Max: Fury Road". One day, both users log in to the platform, expecting personalized recommendations. However, the system suggests an action movie from B's list to A and a science fiction movie from A's list to B. Both users feel dissatisfied and decide to look for content on a different platform. The issue here is that the recommendation system failed to capture the stark differences in their preferences, instead relying solely on their proximity in user-item relationships, leading to inaccurate suggestions.

This raises a crucial question: how can recommendation systems better personalize recommendations, especially when users have entirely different preferences? The answer lies in leveraging the "neighborhood" of each user and understanding the complex relationships between users and items. Rather than relying solely on individual historical data, recommendation systems should utilize information from users with similar interests to create more accurate suggestions. Research shows the significance of this challenge - e-commerce platforms can achieve up to 35% of their revenue from personalized recommendations [3], while Netflix reports that 75% of user viewing choices come from these recommendations [4].

Recommendation systems primarily operate through two main methods: collaborative filtering and content-based filtering [5]. Content-based filtering analyzes item features to suggest similar products based on a user's historical preferences, offering high personalization but often creating "filter bubbles" that limit content diversity [6]. This approach is straightforward to implement as it only requires analyzing item features without extensive user interaction data [7], but struggles significantly with the cold-start problem for new items [8].

Collaborative filtering, conversely, leverages user interaction patterns, assuming users with similar past behaviors will have similar future interests [9]. While this approach enables broader content discovery through neighborhood-based methods and matrix factorization techniques [34], it requires substantial amounts of interaction data and faces challenges with new users or items [10]. Both methods struggle with data sparsity - research from MIT indicates that user-item

interaction matrices often exhibit up to 99.5% sparsity [11], severely impacting recommendation quality.

To address these limitations, researchers developed graph-based approaches using GNNs, which model the recommendation problem as a graph learning task where users and items are nodes connected through various interactions. GNNs can effectively capture both direct and indirect relationships through neighborhood aggregation and message passing mechanisms [12]. This approach has shown remarkable results, with studies from Microsoft Research demonstrating up to 35% improvement in recommendation accuracy compared to traditional methods [13].

However, standard GNN architectures face several critical challenges in recommendation scenarios. First, they suffer from high computational complexity when processing large-scale graphs, with training times increasing quadratically with graph size. Second, the over-smoothing problem in deeper layers leads to indistinguishable node representations, limiting the model's ability to capture fine-grained preferences. Finally, most GNN models treat all relationships uniformly, struggling to handle the heterogeneous nature of real world user-item interactions [14].

To overcome these limitations, we present He-LiGCN, a novel framework that combines multi-level graph coarsening, heterogeneous embeddings, and lightweight convolution operations. The framework incorporates three key innovations: First, it utilizes an intelligent graph coarsening technique that progressively simplifies graph structure while preserving critical relationship patterns, significantly reducing computational complexity. Second, it employs General Attributed Multiplex Heterogeneous Network Embedding (GATNE)-based heterogeneous embeddings [15] that effectively model different types of user-item relationships, enabling more nuanced recommendations. Finally, it introduces lightweight graph convolution operations that maintain model expressiveness while dramatically reducing parameter count and training time.

By returning to our movie platform scenario, He-LiGCN would better serve Users A and B by: efficiently processing their full interaction history through graph coarsening, understanding the distinct nature of their preferences through heterogeneous embeddings, and quickly generating accurate, personalized recommendations through lightweight convolutions. This demonstrates how our approach can address real-world recommendation challenges while maintaining computational efficiency.

2 Related Work

In recent years, graph structures have emerged as a prominent trend in recommendation systems, driven by their significant advantages over traditional data representations. Graphs excel at modeling complex relationships between users, products, and related entities while enabling the simultaneous representation of diverse interactions (views, purchases, ratings) and integration of supplementary information (user characteristics, product attributes) within a unified framework. This creates an ideal environment for applying deep learning techniques, particularly GNNs, to extract complex features and generate automated representations from interaction data.

Representing data as graphs in recommendation systems is not a random choice, but rather a result of the many advantages it offers over traditional data representations [16]. First, the graph structure can naturally and visually model the complex relationships between users, products, and related entities. Graphs also allow the simultaneous representation of multiple types of interactions (such as views, purchases, and ratings) and the integration of additional information (such as user characteristics and product attributes) within a unified representation framework [17]. Moreover, the graph structure can capture indirect relationships through the propagation mechanism, effectively addressing the data sparsity issue - a common challenge in recommendation systems [18]. Graphs also support the discovery of hidden patterns and latent relationships in the data through graph traversal and network structure analysis, which can further improve the quality of recommendations. Most importantly, graph representation provides a foundation for the application of modern graph neural network techniques, opening up new possibilities for learning complex features and representations automatically from interaction data [19].

Graph theory is a branch of discrete mathematics that studies structures called graphs, which are widely used to represent and analyze relationships between entities in many fields, including social networks, e-commerce, and recommendation systems [20]. A graph is defined by two main components: a set of vertices (or nodes) representing entities, and a set of edges representing the relationships between those entities. Graphs can be classified in different ways, such as directed graphs and undirected graphs, and weighted graphs and unweighted graphs [21]. The field originated in the 18th century with Leonhard Euler's solution to the Königsberg Bridge problem, where he proved the impossibility of traversing all bridges exactly once. This work established the concept of graphs and founded modern graph theory, which has since become instrumental in fields ranging from pure mathematics to computer science, network research, and machine learning.

In the context of recommendation systems, graphs provide a powerful approach to modeling user-item interactions [22]. In a user-product graph, vertices represent users and products, while edges represent interactions like purchasing, rating, or adding to cart. These graph structures carry rich semantic information about entity relationships, enabling analysis and pattern discovery in the data [23]. A

fundamental concept in this representation is vertex degree - the number of edges connected to a vertex [24]. Degree metrics provide important insights about entity importance within the network, such as identifying influential users with many connections in social recommendation scenarios [25].

Based on the properties of the vertices and edges, graphs can be categorized into two main types: homogeneous graphs and heterogeneous graphs [26]. Homogeneous graphs are the simplest type, where all vertices and edges belong to the same type. This type of graph is often used to model systems with a homogeneous structure, such as a social network where the vertices represent users and the edges represent friendship relationships [27]. The homogeneity of homogeneous graphs makes them easy to represent and process, allowing the efficient application of basic graph analysis algorithms. However, the simplicity of this graph type also poses a significant limitation when facing complex systems with diverse entities and relationships.

In contrast, heterogeneous graphs are designed to model more complex systems, where the vertices and edges belong to multiple different types [28]. In an e-commerce system, for instance, a heterogeneous graph may include different types of vertices such as users, products, and categories, while the edges represent relationships like purchasing, reviewing, or belonging to a category [29]. The flexibility of heterogeneous graphs provides a superior ability to model complex relationships and offers a more comprehensive view of the system's structure. However, processing heterogeneous graphs requires more advanced algorithms and incurs higher computational costs than homogeneous graphs [30].

Reducing the size of the graph before performing embeddings is an important step in many graph processing tasks, especially when working with large or complex graphs [31]. The main reason for this is to reduce computational complexity, improve model performance, and mitigate memory overload when processing the data. When the graph has too many vertices and edges, computing embeddings for each element can be computationally expensive, especially for deep learning algorithms that optimize millions of parameters [32]. Therefore, graph reduction helps decrease the number of elements that need to be processed, making the learning process faster and more efficient.

In the process of graph processing, each graph reduction method has its own advantages and limitations, depending on the specific characteristics of the problem [33]. Graph sampling, which randomly selects a small portion of the graph, helps reduce computational cost when working with large graphs. However, the downside of this method is that random sampling may lead to the loss of important information from the unselected parts, resulting in reduced accuracy for applications requiring high precision [34]. Graph coarsening, on the other hand, solves this problem by merging strongly connected vertices, helping to maintain important features while reducing the graph size [33]. However, this method also has limitations, as it may lose subtle structural details, especially in complex and diverse graphs. Graph pruning, another method, removes unimportant vertices or edges to reduce graph complexity [35]. While this helps reduce computational cost, if not done carefully, this approach may also remove important latent

relationships. Dimensionality reduction, on the other hand, reduces the feature dimensionality without losing critical information, but it can also discard complex relationships in the data, especially when the data contains many features that are not easily representable in low-dimensional spaces [36].

Particularly for heterogeneous graphs, where vertices and edges can belong to multiple different types, reducing complexity while maintaining the diverse and complex relationships between the components is a major challenge [28]. Methods like graph coarsening or community detection are effective in reducing graph complexity, but they struggle to maintain the diverse and complex structure of the relationships between different types of vertices and edges. This is why Multi-Level Embedding framework of nodes on a heterogeneous graph (HeteroMILE) becomes an optimal method for our problem [37]. Unlike traditional approaches, HeteroMILE not only helps reduce complexity but also optimizes the embedding process for heterogeneous graphs, where vertices and edges belong to multiple different types. HeteroMILE helps preserve the important relationships between these components, especially in tasks that require high accuracy.

By leveraging specialized techniques to learn embeddings for the diverse vertex and edge types, HeteroMILE not only reduces the graph size but also retains the critical information from the complex relationships [38]. This is extremely important for tasks that require high accuracy in the associations between different objects in the graph. Furthermore, HeteroMILE allows the optimization of the graph in the embedding space to be performed smoothly without losing necessary information, something that other methods like graph pruning or dimensionality reduction struggle to achieve when working with complex and heterogeneous graphs [39].

An important factor in graph analysis is the representation of vertices and edges in vector space through graph embedding techniques [40]. Graph embedding is the process of transforming the components of a graph, including both vertices and edges, into continuous vector representations. These vectors not only help reduce the data dimensionality but also preserve the important structural and contextual properties, enabling their use in machine learning models or prediction tasks [41]. This process is particularly crucial when working with large and complex graphs, where direct processing of the connectivity information becomes infeasible.

Graph embedding is applied to both homogeneous and heterogeneous graphs. In homogeneous graphs, the embedding learning typically focuses on local properties, such as vertex degree or relationships between neighboring vertices [42]. In contrast, heterogeneous graphs require the integration of information from multiple different vertex and edge types, necessitating more sophisticated algorithms like GATNE or Heterogeneous Graph Convolutional Network (HGCN) [43].

DeepWalk was one of the first and groundbreaking methods for learning graph embeddings [23]. This algorithm is inspired by natural language processing, where graph vertices are treated as words and random walk sequences as sentences. Specifically, the algorithm performs random walks from each vertex in the graph with a predefined length. These sequences are considered the local contexts of the vertices, which are then processed through a Skip-Gram model to learn the vector

representations for each vertex. However, DeepWalk has significant limitations, such as not leveraging additional information about edge weights or directions, and the random walks may miss important global-level information [44].

To address the limitations of DeepWalk, Node2Vec was developed with the ability to control the exploration strategy in the random walk sequences through two parameters, p and q [45]. The parameter p governs the likelihood of revisiting a previously visited vertex, while q determines the tendency to explore more distant vertices. By combining depth-first and breadth-first exploration strategies, Node2Vec achieves an optimal balance between local and global information [46].

GraphSAGE introduced a new approach by using an aggregation strategy to learn embeddings [47]. The key difference of GraphSAGE is its ability to generate dynamic embeddings that can be recomputed based on the current information. The algorithm works by sampling neighboring vertices and aggregating information from these vertices to create new embeddings. This approach allows the application to large graphs and the handling of new vertices, although it requires higher computational costs [48].

While GraphSAGE focuses on aggregating information from neighboring nodes in homogeneous graphs, new challenges arise when dealing with heterogeneous graphs, driving the development of more advanced methods [49]. GATNE (General Attributed Multiplex Heterogeneous Network Embedding) is an important step forward in handling heterogeneous graphs [15]. Unlike traditional graph embedding methods that treat all relationships equally, GATNE effectively integrates different types of edge information by using type-specific feature transformations and an attention mechanism [50]. This model learns context-aware embeddings, considering the specific nature of each relationship type, allowing it to capture the diversity of interactions in a multi-relational graph. GATNE represents each node by combining a base embedding (capturing common features across all edge types) with type-specific embeddings through an attention mechanism [51]. This approach helps the model learn rich structural and attribute information from the different edge types, while maintaining computational efficiency through parameter sharing and attention-based aggregation.

HGCN extends the capabilities of traditional GCN to handle heterogeneous graphs, addressing the challenge of effectively modeling the diverse node and edge types in a heterogeneous graph for recommendation tasks [52]. HGCN extends the standard GCN framework by introducing type-specific transformations and an attention mechanism that can capture the semantic relationships between different node types. The model uses a heterogeneous graph convolution approximation, where node representations are updated by aggregating information from neighboring nodes of different types through normalized attention coefficients [53]. This design allows HGCN to preserve the distinctive features of each node type while still enabling efficient information propagation across the heterogeneous graph structure. The attention mechanism in HGCN helps determine the most relevant relationships for each node, improving both the efficiency and interpretability of the model in capturing complex heterogeneous relationships [14].

GNNs have marked an important step in processing graph data with their efficient information propagation mechanisms between vertices [54]. The evolution of GNNs in recommendation systems has seen several significant milestones, from traditional GCN to more sophisticated models. The Neural Collaborative Filtering (NCF) [55] framework first demonstrated the potential of neural networks in recommendation systems by replacing linear interactions with non-linear neural architectures. Building upon this foundation, Neural Graph Collaborative Filtering (NGCF) [56] emerged as a crucial advancement by explicitly incorporating the user-item graph structure into the embedding learning process. NGCF leveraged high-order connectivity by propagating embeddings on the user-item interaction graph, which helped capture collaborative signals more effectively than traditional methods.

Among these developments, Lightweight Graph Convolutional Network (LightGCN) [57] stands out as a breakthrough improvement, developed with a "simplify to the maximum" philosophy while maintaining high performance. Unlike traditional GCN models that often use many complex components such as non-linear functions, feature transformation matrices, and self-connection mechanisms, LightGCN demonstrates that these components are not always necessary for recommendation tasks [56]. While NGCF incorporated various operations including feature transformation and non-linear activation, LightGCN proposes a simple yet effective information propagation mechanism between neighboring nodes, combined with a multi-layer aggregation approach. This minimalist design brings several significant advantages [57]. First, reducing the number of parameters and simplifying the architecture significantly decreases the computational cost and speeds up the training process. Second, the reduced complexity helps mitigate overfitting and improves generalization. Notably, experiments have shown that textsfLightGCN not only is computationally efficient but also achieves higher accuracy than complicated GCN models in many recommendation tasks. However, LightGCN does have certain limitations. Due to its simple design, the model may struggle to capture complex or non-linear relationships between nodes. Additionally, the lack of non-linear transformations can constrain the model's representational capability in some specific cases. Nevertheless, these limitations are often outweighed by the overall superior performance of the model in most real-world applications [57].

In addition to embedding methods, an important challenge in applying GNNs is the phenomenon of over-smoothing, Approximate Personalized Propagation of Neural Predictions (APPNP) [58] is introduced as an improvement to address this issue. This problem occurs when the representations of the nodes in the graph become increasingly similar after multiple steps of information propagation, leading to a loss of distinguishability between nodes [59]. APPNP applies a personalized information propagation mechanism that combines maintaining the original information of each node with the aggregation of information from neighboring nodes, helping to balance the exploitation of local and global information [60]. As a result, APPNP not only improves accuracy in tasks that require high distinguishability but also mitigates the impact of sparse data. With good scalability and high computational efficiency, APPNP has demonstrated its outstanding potential in applications such as recommendation systems, where

personalization and handling large graphs are key factors [61].

3 Methodology

To overcome the scalability and accuracy limitations inherent in traditional recommendation systems, this study introduces He-LiGCN, a framework designed to improve recommendation effectiveness through advanced heterogeneous graph modeling techniques. The He-LiGCN framework in Figure 3.1 is built upon four key components:

- Multi-level graph coarsening: Reduces computational complexity by iteratively simplifying the graph structure while preserving critical relationships.
- GATNE: Captures complex structural and attribute information across multiple types of nodes and edges, enabling a richer representation of user-item interactions.
- HGCN: Refines node embeddings by integrating both local and global graph information, enhancing the expressiveness of the model.
- Light Graph Convolution (LGC): Efficiently aggregates neighborhood information across multiple graph layers, ensuring scalability and high accuracy even in large-scale, heterogeneous datasets.

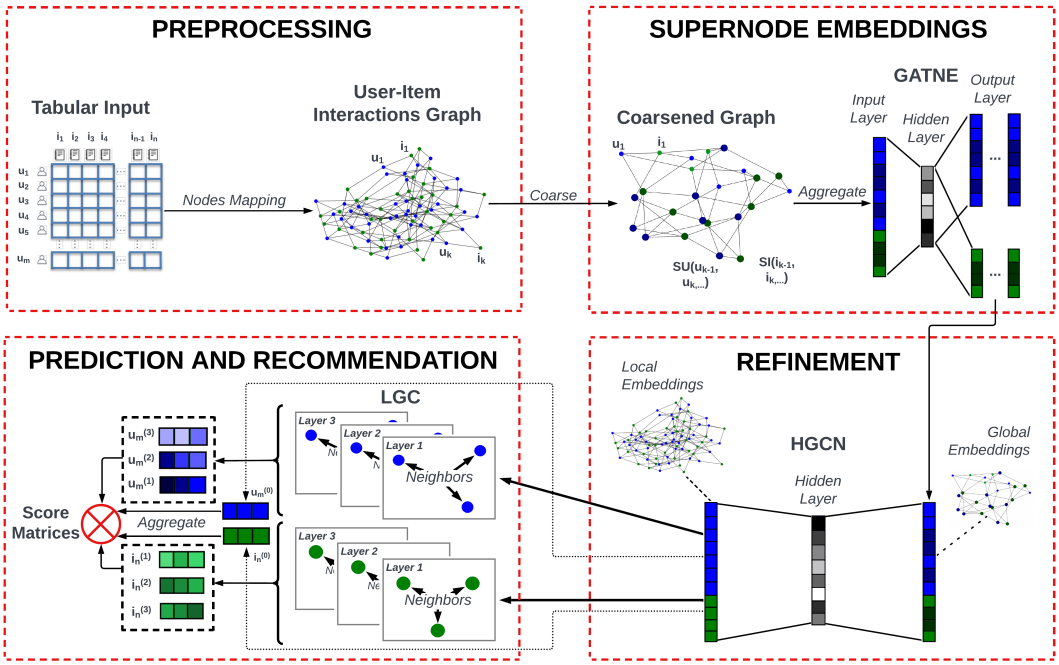


Figure 3.1: The He-LiGCN workflow.

Figure 3.1 provides an overview of the He-LiGCN workflow, detailing the primary stages: data preprocessing, supernode embedding generation, embedding refinement, and final prediction and recommendation. The integration of GATNE and HGCN enables the model to construct both global and local embedding representations, while LGC facilitates the effective aggregation of supernodes and user-item interactions across multiple neighborhood layers. This cohesive design ensures

that He-LiGCN can deliver accurate, personalized recommendations in complex, large-scale environments.

3.1 Preprocessing: Constructing Efficient Heterogeneous Graphs

The preprocessing phase is essential for converting raw user-item interaction data into a structured heterogeneous graph that can be efficiently utilized by the He-LiGCN framework. This phase involves three primary steps: constructing the initial heterogeneous graph from tabular input (Figure 3.2), mapping nodes to get user-item interaction graph, and applying graph coarsening techniques to reduce complexity while retaining critical structural information. Two key techniques used for graph coarsening are Jaccard Index [62] and Locality Sensitive Hashing (LSH) [63]:

- Jaccard Index measures the overlap between neighboring nodes, helping identify and merge similar nodes based on their 1-hop and 2-hop neighborhoods.
- LSH complements this by grouping similar nodes into the same "buckets," enabling faster and more efficient node merging in large-scale graphs.

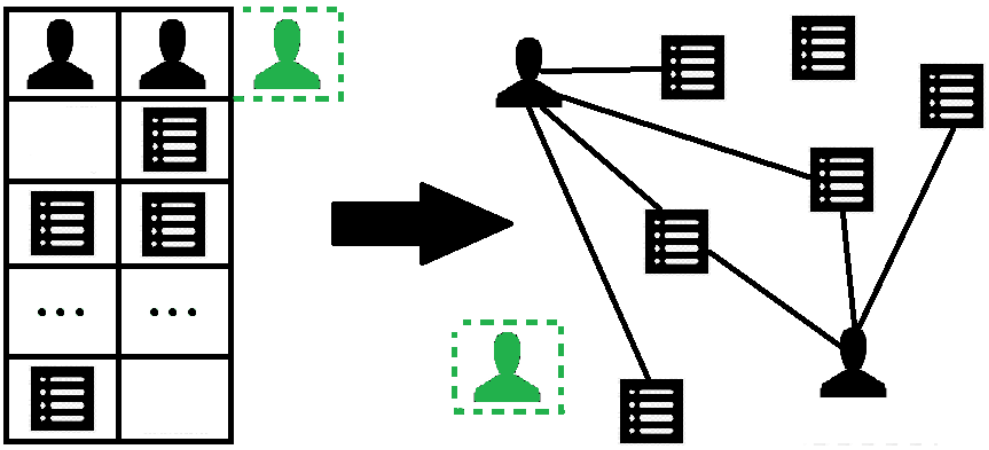


Figure 3.2: Initialized heterogeneous graph process.

3.1.1. Node Similarity with Jaccard Index

The coarsening process begins with an initial heterogeneous graph $\mathcal{G}_0$, which is iteratively reduced into a sequence of coarsened graphs $\mathcal{G}_1, \ldots, \mathcal{G}_m$, with $\mathcal{G}_m$ being the simplest form. In Figure 3.3, at each level i, nodes in graph $\mathcal{G}_i$ are matched and merged to form $\mathcal{G}_{i+1}$. This hierarchical reduction significantly reduces the graph's size, thereby improving computational efficiency while preserving essential structural patterns for downstream tasks.

In this step, the Jaccard Index is used to measure the similarity between nodes by evaluating the overlap of their 1-hop and 2-hop neighbors. This metric captures how similar two nodes are based on their immediate surroundings, providing a

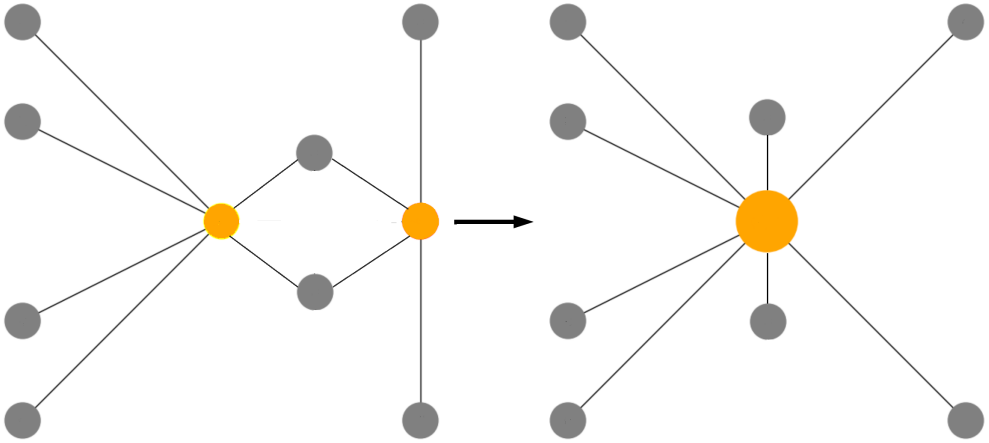


Figure 3.3: Coarsening process.

foundation for merging nodes into supernodes in the coarsening process. Two strategies are employed for node merging based on Jaccard Similarity.

Jaccard Max: For each unmatched node u, a node v is selected from $\mathcal{N}_u$ based on the highest Jaccard similarity score. Node u and v are then merged into a single supernode. This process continues iteratively until all nodes are matched or no suitable neighbors remain. The Jaccard Similarity is calculated as

$$J(u,v) = \frac{|\mathcal{N}_u \cap \mathcal{N}_v|}{|\mathcal{N}_u \cup \mathcal{N}_v|} = \frac{|\mathcal{N}_u \cap \mathcal{N}_v|}{|\mathcal{N}_u| + |\mathcal{N}_v| - |\mathcal{N}_u \cap \mathcal{N}_v|} \tag{3.4}$$

This approach ensures that nodes with the most overlapping neighborhoods are merged first, preserving key structural relationships.

Jaccard Weighted Random Sampling (WRS): This method selects a neighboring node v_j from $\mathcal{N}_u$ randomly, with the selection probability proportional to their Jaccard similarity score. The probability of merging u with v_j is defined as

$$p(u,v_j) = \frac{J(u,v_j)}{\sum_{i \in \mathcal{N}_u} J(u,v_j)} \tag{3.5}$$

This probabilistic approach introduces diversity in the coarsening process, reducing the risk of suboptimal local merges and enhancing global graph structure.

3.1.2. Locality Sensitive Hashing: More Efficient Node Grouping

LSH provides an efficient approach to grouping nodes with similar neighborhoods in Figure 3.6 by hashing them into the same "bucket." Nodes with similar neighbors are more likely to share the same hash value, making LSH an effective technique for identifying approximate nearest neighbors in high-dimensional data.

The LSH process works by assigning each node a signature vector, where each component is defined by a hash function f_i. For two nodes u and v, the similarity

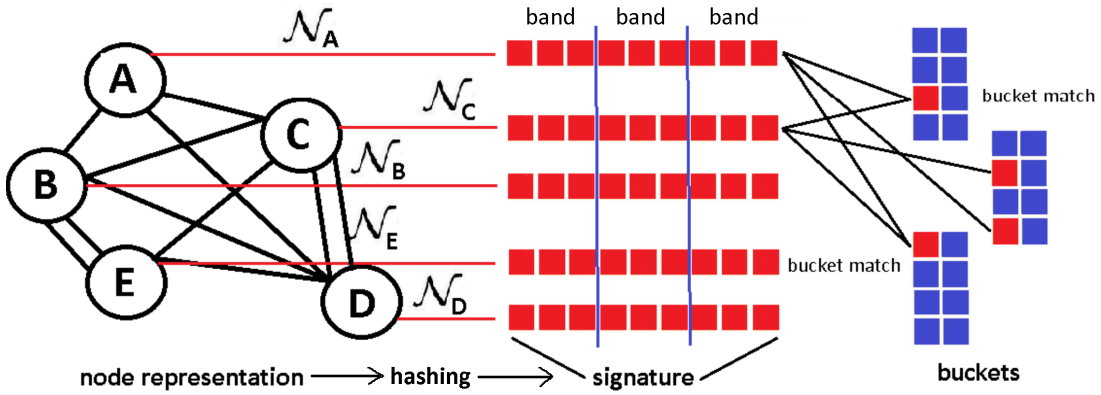


Figure 3.6: LSH process.

between their neighborhoods $\mathcal{N}_u$ and $\mathcal{N}_v$ is estimated by the probability that they share the same hash value across multiple hash functions

$$\text{sim}(\mathcal{N}_u, \mathcal{N}_v) = P[f_i(\mathcal{N}_u) = f_i(\mathcal{N}_v)] \tag{3.7}$$

Here, each hash function f_i maps a node u to the minimum value of a random permutation $\pi(u)$

$$f_i(u) = \min \pi(u) \tag{3.8}$$

The probability that two nodes u and v share the same hash value is proportional to their Jaccard similarity in equation 3.2

$$P[f_i(u) = f_i(v)] = J(u, v) \tag{3.9}$$

Compared to the traditional Jaccard similarity calculation, which has a time complexity of $O(|\mathcal{V}^2|)$ for pairwise node comparisons in large graphs, LSH significantly reduces the complexity to $O(|\mathcal{V}|)$ per iteration. This improvement is achieved by hashing nodes into buckets based on their neighbor similarity, allowing the algorithm to focus only on nodes within the same bucket instead of comparing all node pairs. By avoiding exhaustive pairwise comparisons, LSH enhances scalability and makes it suitable for processing large-scale, high-dimensional graphs.

3.2 Supernode Embedding

The graph coarsening process significantly reduces the size of the graph, often halving its complexity at each iteration. After m iterations, the graph is reduced to its coarsest form, $\mathcal{G}_m$, which serves as the input for the embedding generation process. In this phase, GATNE technique is employed to generate initial node embeddings on $\mathcal{G}_m$. This method captures multiplex relationships and structural diversity across different types of nodes and edges, enabling a comprehensive representation of the graph. To further enhance the quality of these embeddings, a customized loss function is applied, optimizing the model to better capture both local and global dependencies. This combined approach ensures that each supernode accurately reflects the complex, heterogeneous nature of the original graph while maintaining computational efficiency for downstream tasks.

3.2.1. Generating Initial Embeddings with General Attributed Multiplex Heterogeneous Network Embedding

In our implementation of He-LiGCN, we leverage GATNE framework as a crucial component for generating initial embeddings on the coarsened graph $\mathcal{G}_m$. We selected GATNE specifically for its sophisticated ability to handle the complex, heterogeneous nature of user-item interaction networks, where multiple types of relationships coexist and interplay with each other.

In Figure 3.10, the embedding generation process in GATNE operates through a carefully designed three-layer architecture. The input layer initializes node embeddings by incorporating both basic node attributes and relationship-specific transformation matrices. This foundation allows us to capture the inherent characteristics of each node while preserving the distinctive features of different interaction types. The hidden layer then aggregates information from neighboring nodes, using an attention mechanism to weigh the importance of different edge types. This attention-based approach enables the model to adaptively focus on the most relevant relationships for each node, significantly improving the quality of learned representations.

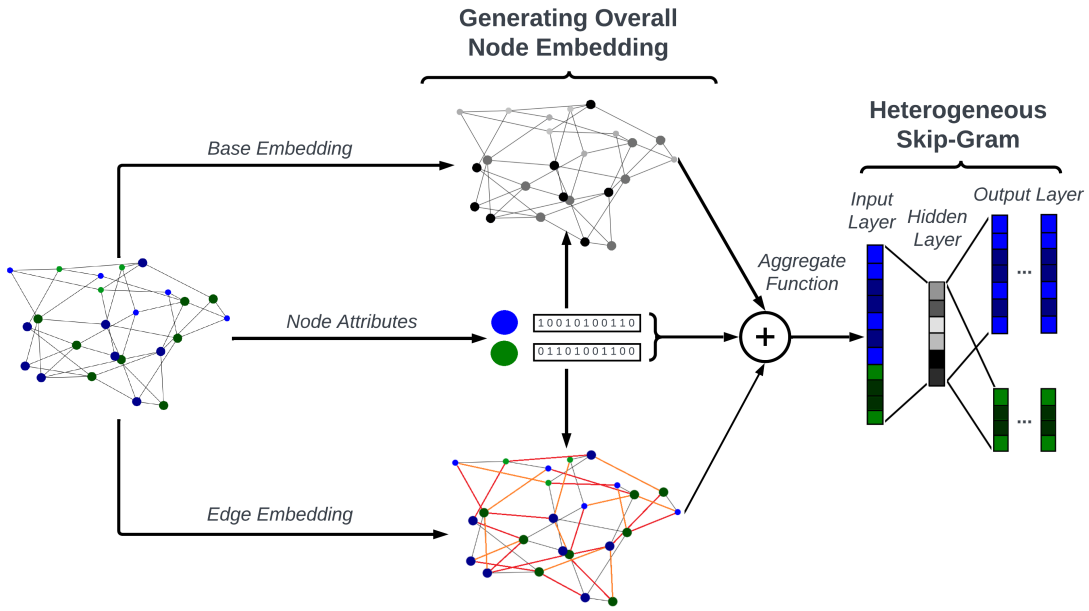


Figure 3.10: The components of the GATNE model.

Building upon these initial representations, the output layer generates the final embeddings that capture the multiplex relationships across various edge types. This layered approach ensures our embeddings reflect both the structural and semantic aspects of the graph, providing a robust foundation for subsequent refinement steps.

To optimize these embeddings, we employ a heterogeneous skip-gram objective that learns from both positive and negative samples. The optimization process uses stochastic gradient descent with carefully tuned hyperparameters: η for controlling the learning rate, L for determining the number of negative samples,

and α, β for balancing the influence of edge-type-specific and attribute-based components respectively.

The GATNE model operates on $\mathcal{G}$, the embeddings are learned in a d-dimensional space, with additional edge-type-specific embeddings in an s-dimensional space. To effectively capture the influence of various relationships, the model utilizes key hyperparameters η (learning rate for the embedding optimization), L (number of negative samples for contrastive learning), α (weight factor for edge-type-specific embeddings), and β (weight factor for attribute-based embeddings). The embedding process begins by initializing the parameter set θ, including base embeddings for each node; transformation matrices for each edge type to account for type-specific relationships; attention coefficients that determine the importance of different neighbors during aggregation; context embeddings that represent the interaction context between nodes.

To capture the local structural properties of each edge type, the model performs random walks separately for each type of edge r. These random walks explore the graph structure and generate sequences of nodes that reflect the local neighborhood of each target node. In Figure 3.11, for each target node v_i, a sequence of training samples (v_i, v_j, r) is generated, where v_j appears within a predefined window size of v_i during the random walk. This sampling strategy ensures that the model learns edge-type-specific patterns and relationships, capturing both direct and indirect interactions between nodes. The resulting training samples are then used to optimize the embeddings, enabling the model to represent nodes with respect to both their structural roles and their contextual importance within the graph.

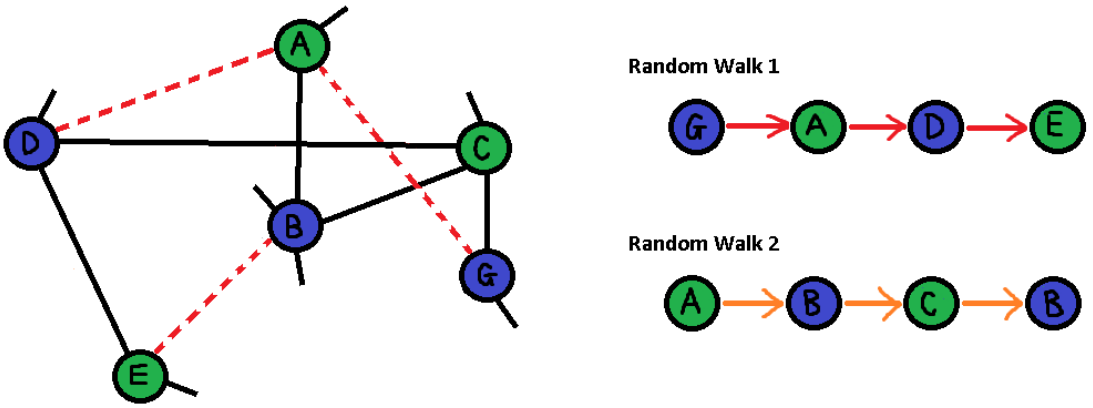


Figure 3.11: Random walk implementation.

The core of GATNE model lies in the iterative optimization process, which continues until convergence. For each training sample, the model calculates $\boldsymbol{v}_{i,r}$ for v_i. This embedding is computed depending on whether node attributes are being incorporated. If node attributes are not used (transductive), $\boldsymbol{v}_{i,r}$ is calculated as

$$\boldsymbol{v}_{i,r} = \boldsymbol{b}_i + \alpha_r \boldsymbol{M}_r^\top \boldsymbol{U}_i \boldsymbol{a}_{i,r} \tag{3.12}$$

where

$$\boldsymbol{U}_i \in \mathbb{R}^{s \times m} = \boldsymbol{u}_{i,1}^{(K)} \,||\, \boldsymbol{u}_{i,2}^{(K)} \,||\, \cdots \,||\, \boldsymbol{u}_{i,m}^{(K)} \tag{3.13}$$

and

$$\boldsymbol{a}_{i,r} \in \mathbb{R}^m = \text{softmax}(\boldsymbol{w}_r^\top \tanh(\boldsymbol{W}_r \boldsymbol{U}_i))^\top \tag{3.14}$$

$\boldsymbol{b}_i$ capturing general features across all edge types, α_r controlling the influence of the edge-type-specific component, $\boldsymbol{u}_{i,1}^{(K)} \in \mathbb{R}^s$ is the level edge embeddings of v_i on edge type r, aggregated from $\mathcal{N}_{i,r}$.

When node attributes are included (inductive), $\boldsymbol{v}_{i,r}$ is computed as

$$\boldsymbol{v}_{i,r} = \boldsymbol{h}_z(\boldsymbol{x}_i) + \alpha_r \boldsymbol{M}_r^\top \boldsymbol{U}_i \boldsymbol{a}_{i,r} + \beta_r \boldsymbol{D}_z^\top \boldsymbol{x}_i \tag{3.15}$$

In this equation, $\boldsymbol{h}_z(\boldsymbol{x}_i)$ often implemented using a neural network to capture complex attribute interactions, the additional term $\beta_r \boldsymbol{D}_z^\top \boldsymbol{x}_i$ introduces an attribute-specific component.

The strength of GATNE in our framework lies in its ability to handle both transductive and inductive scenarios. In the transductive setting, node embeddings are computed as a combination of base embeddings and edge-type-specific components. For inductive tasks, we incorporate node attributes through a neural network transformation, enabling our model to generalize to previously unseen nodes.

This comprehensive approach to embedding generation has proven particularly effective in our recommendation system context, where understanding the nuanced relationships between users and items is crucial for making accurate predictions. The resulting embeddings serve as a strong foundation for the subsequent refinement phases of our He-LiGCN framework.

3.2.2. Loss Function

After the computation, the model proceeds to optimize it by considering both positive and negative samples. It samples L negative nodes $\{v_k\}$ from a noise distribution $P_t(v)$, representing nodes that are unlikely to co-occur with v_i under edge type r. The objective function $-\log P_\theta(\{v_j \mid v_j \in C\} \mid v_i)$ for each pair (v_i, v_j) is approximated as cross-entropy loss function [64]

$$\mathcal{E} = \sum_{v_j \in \mathcal{C}} -\log\left(\frac{e^{\boldsymbol{c}_j^\top \boldsymbol{v}_{i,r}}}{\sum_{k \in \mathcal{V}_t} e^{\boldsymbol{c}_k^\top \boldsymbol{v}_{i,r}}}\right) \approx -\log \sigma(\boldsymbol{c}_j^\top \boldsymbol{v}_{i,r}) - \sum_{l=1}^{L} \mathbb{E}_{v_k \sim P_t(v)} \left[\log \sigma(-\boldsymbol{c}_k^\top \boldsymbol{v}_{i,r})\right] \tag{3.16}$$

where σ is the sigmoid function. $\mathcal{E}$ aims to maximize the similarity between the embedding of target node and its true context node while minimizing the similarity with negative samples. The first term represents the loss associated with the positive sample, encouraging the model to assign higher similarity scores to true context nodes, while the second term penalizes the model if negative samples are assigned high similarity scores, thereby pushing the embeddings of unrelated nodes apart.

To update θ, $\frac{\partial E}{\partial \theta}$ are computed, and the parameters are adjusted in the direction that minimizes $\mathcal{E}$. This is typically done using stochastic gradient descent (SGD) [65] or its variants. The model gradually learns embeddings that capture both the structural patterns and attribute information in the network iteratively.

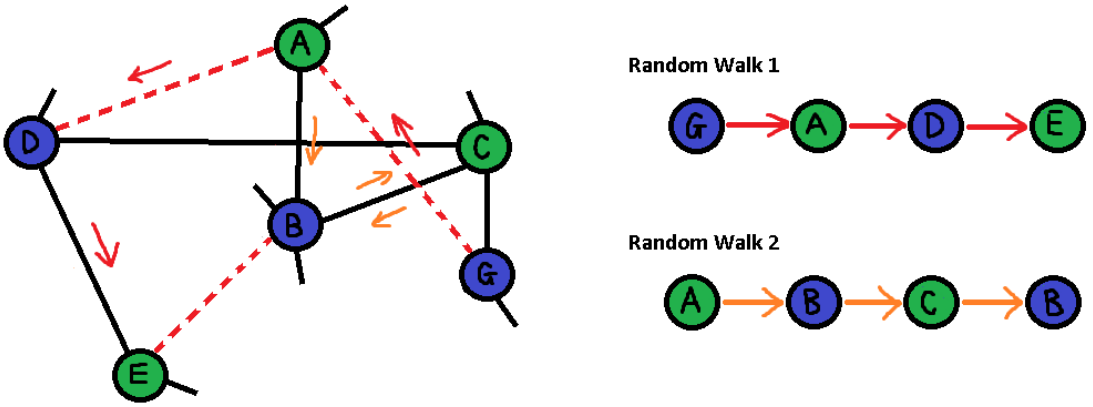


Figure 3.17: Example of negative sample (random walk 1) and positive samples (random walk 2).

GATNE is a comprehensive model that addresses the challenges of embedding nodes in multiplex heterogeneous networks. It does so by initializing model parameters, generating edge-type-specific random walks, creating training samples from these walks, and iteratively optimizing the embeddings through gradient-based methods. The model meticulously balances the structural and attribute information through the use of attention coefficients and transformation matrices, resulting in embeddings that are both informative and adaptable to different tasks.

3.3 Refinement

After generating the initial supernode embeddings, the refinement phase aims to enhance these embeddings by accurately capturing both local and global structural information within the graph. This step is critical for improving the expressiveness and accuracy of the node representations, especially in complex recommendation tasks where user-item interactions span multiple levels of the graph.

To achieve this, we utilize HGCN, which iteratively refines the embeddings through a multi-layer convolutional process. The primary objective is to progressively reconstruct the embeddings of the original graph $\mathcal{G}_0$ from the coarsened graph $\mathcal{G}_m$, ensuring that the refined embeddings retain the essential structural characteristics of the original graph. The refinement process starts by inferring the embeddings $\boldsymbol{E}_i$ for each intermediate graph $\mathcal{G}_i$ using the embeddings from its coarser version $\mathcal{G}_{i+1}$, along with the matching matrix $\boldsymbol{M}_{i,i+1}$, which maps nodes between the two graphs. This hierarchical approach allows for a gradual and systematic reconstruction of the original node embeddings.

After completing this step, we systematically apply the same approach to successive graph pairs, starting from $\mathcal{G}_m$ and moving backward to $\mathcal{G}_0$, gradually obtaining the node embeddings. To achieve this, we employ a HGCN model, which helps refine and enhance the quality of the embeddings.

3.3.1. Enhancing Representations with Heterogeneous Graph Convolutional Networks

The refinement process with HGCN is designed to enhance the quality of supernode embeddings by incorporating both local and global information from the graph structure. This is achieved through a series of convolutional operations that propagate information across different layers of the graph. Using the matching matrix $\boldsymbol{M}_{i,i+1}$, the embeddings from the coarser graph $\mathcal{G}_{i+1}$ are projected onto the corresponding nodes in the finer graph $\mathcal{G}_i$, allowing for a smooth transition between different levels of granularity. The projection is computed as

$$\boldsymbol{E}_i' = \boldsymbol{M}_{i,i+1}\boldsymbol{E}_{i+1} \tag{3.18}$$

The projected embeddings $\boldsymbol{E}_i'$ retain structural information from the coarser graph while adapting it to the finer graph's context, enabling a more accurate representation of the original nodes. This step ensures that the refined embeddings not only capture the overall structure of the graph but also preserve important local interactions.

Figure 3.19 highlights how embeddings from higher-level graph structures are projected and aggregated into lower-level structures, enabling multi-scale representation learning within heterogeneous graphs.

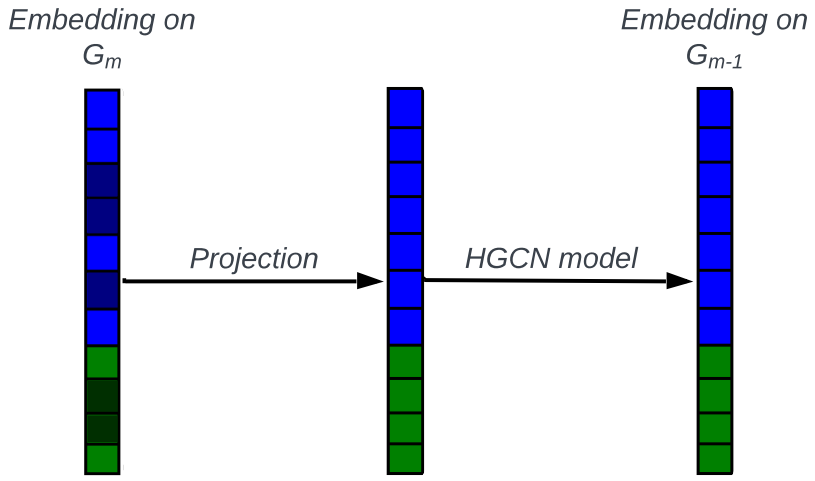


Figure 3.19: The process of embedding generation for two layers, G_m and G_{m-1}, using hierarchical projection and representation learning.

Supernode embeddings are replicated for each related original node. $\boldsymbol{E}_i'$ capture structural information from the coarser graph and represent nodes in the refined embedding space. Figure 3.20 below showcases the generation of global embeddings, which captures the overall structure of the graph, and local embeddings, which represents fine-grained neighborhood interactions. The model's hidden layers aggregated these embeddings to learn representations that balances local and global information for heterogeneous graphs.

The refinement process in HGCN updates node embeddings by propagating information from both the target nodes and their heterogeneous neighbors. This is

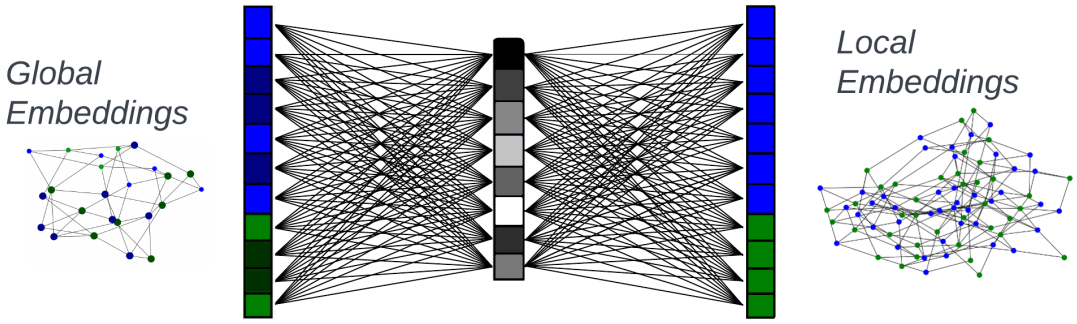


Figure 3.20: The architecture of HGCN.

achieved through a heterogeneous convolution operation that combines information from multiple node types and edge relationships. The embeddings for the next layer are computed as

$$\boldsymbol{H}_i^{O_i'} = \sigma\left(\boldsymbol{\Phi}_i^{\langle O_i\rangle} \cdot \boldsymbol{Z}_{i,:}^{\langle O_i\rangle} + \sum_{O_{\mathcal{N}} \in \mathcal{N}_{O_i}} \boldsymbol{\Phi}_i^{\langle O_{\mathcal{N}} \to O_i\rangle} \cdot \boldsymbol{Z}_{i,:}^{\langle O_{\mathcal{N}} \to O_i\rangle}\right) \tag{3.21}$$

In this formula, $\boldsymbol{H}^{O_i}$ is the refined embedding of node i of type O_i, σ represents the non-linear activation function, such as LeakyReLU [66], to introduce non-linearity and enhance model expressiveness, $\boldsymbol{\Phi}_i^{\langle O_i\rangle}$ and $\boldsymbol{\Phi}_i^{\langle O_{\mathcal{N}} \to O_i\rangle}$ is attention coefficients determining the importance of the node's self-information and the neighboring node type $O_{\mathcal{N}}$ respectively.

The term $\boldsymbol{Z}^{\langle O_i\rangle}$ aggregates information from neighboring nodes

$$\boldsymbol{Z}^{\langle O_i\rangle} = \boldsymbol{H}^{O_i} \cdot \boldsymbol{W}^{\langle O_i\rangle} \tag{3.22}$$

where $\boldsymbol{W}^{\langle O_i\rangle}$ is a learnable weight matrix specific to node type O_i. This transformation projects the node embeddings into a shared latent space, allowing for meaningful aggregation with other node types.

The term $\boldsymbol{Z}^{\langle O_{\mathcal{N}} \to O_i\rangle}$ aggregates information from neighboring nodes

$$\boldsymbol{Z}^{\langle O_{\mathcal{N}} \to O_i\rangle} = \hat{\boldsymbol{A}}^{\langle O_i - O_{\mathcal{N}}\rangle} \cdot \boldsymbol{H}^{O_{\mathcal{N}}} \cdot \boldsymbol{W}^{\langle O_{\mathcal{N}} \to O_i\rangle} \tag{3.23}$$

where $\hat{\boldsymbol{A}}^{\langle O_i - O_{\mathcal{N}}\rangle}$ is the normalized adjacency matrix representing the connections between nodes of type $O_{\mathcal{N}}$ and O_i, $\boldsymbol{H}^{O_{\mathcal{N}}}$ is the embedding matrix of neighboring nodes of type $O_{\mathcal{N}}$, $\boldsymbol{W}^{\langle O_{\mathcal{N}} \to O_i\rangle}$ is the learnable weight matrix for the edge type connecting $O_{\mathcal{N}}$ to O_i.

This convolution operation aggregates information across different types of nodes and edges, allowing the model to capture both intra-type (within the same node type) and inter-type (across different node types) relationships. Finally, the output embeddings $\boldsymbol{H}_i^{O_i'}$ for each layer are passed to the next layer for further refinement, progressively improving the model's ability to capture complex patterns in the heterogeneous graph.

3.3.2. Loss Function

The loss function in this phase is crucial for refining the quality of node embeddings by minimizing the discrepancy between the predicted embeddings and their ground-truth counterparts. The primary objective is to ensure that the refined embeddings accurately capture the structural and semantic properties of the original graph, enhancing the model's predictive capabilities in recommendation tasks.

To achieve this, we employ the Mean Squared Error (MSE) [67] loss function, which measures the difference between the predicted embeddings $\boldsymbol{E}_i$ and the ground-truth embeddings $\boldsymbol{H}_i$. The loss is calculated as follows

$$\mathcal{L} = \frac{1}{|\mathcal{V}_i|} \sum_{i=1}^{|\mathcal{V}_i|} \|\boldsymbol{E}_i - \boldsymbol{H}_i\|^2 \tag{3.24}$$

Here, $\boldsymbol{E}_i$ represents the refined embedding of node i after the convolutional refinement process, $\boldsymbol{H}_i$ denotes the ground-truth embedding of node i, which is derived from the previous layer or from the original input embeddings.

This loss function penalizes large discrepancies between the predicted and ground-truth embeddings, driving the model to produce more accurate representations. By minimizing the MSE, the model learns to preserve essential graph properties such as neighborhood relationships and node similarities, which are critical for generating effective recommendations.

To optimize the loss function, we utilize SGD with adaptive learning rate strategies such as Adam [68]. These optimizers allow for efficient convergence even in large-scale graphs with high-dimensional embeddings. The model parameters, including the weight matrices and attention coefficients, are iteratively updated to minimize the loss, gradually improving the quality of the embeddings at each layer.

3.4 Prediction and Recommendation

The final phase of the He-LiGCN framework focuses on aggregating refined embeddings and generating accurate predictions for recommendation tasks. At this stage, it is crucial to combine information from multiple layers of the graph to capture both direct and indirect interactions between users and items. By aggregating these interactions, the model can better understand user preferences and item relationships, leading to more precise recommendations.

To achieve efficient and scalable information aggregation, we employ LGC in Figure 3.25, a lightweight yet effective technique for combining multi-layer neighborhood information without introducing significant computational overhead. After aggregation, the user-item interactions are represented as a matrix, which serves as the basis for generating recommendation scores. Finally, a custom loss function is applied to optimize the recommendation performance, ensuring that the predicted scores align with actual user preferences.

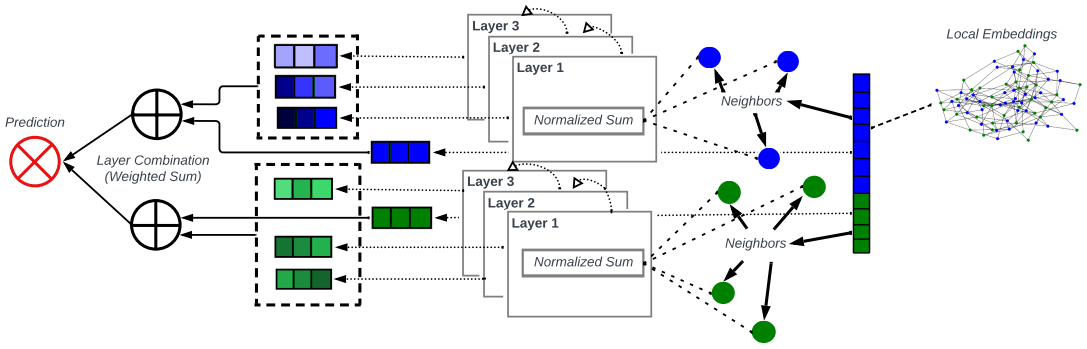


Figure 3.25: The architecture of LGC model.

3.4.1. Efficient Information Aggregation with Light Graph Convolution

LGC plays a pivotal role in aggregating information across multiple graph layers. LGC is designed to efficiently combine neighborhood information without the need for complex feature transformations or non-linear activations. By focusing purely on the propagation of useful information between nodes, LGC significantly reduces the computational burden while maintaining the quality of the aggregation process.

This method involves aggregating the neighborhood information for each node in the graph layer by layer, progressively building richer embeddings as the graph layers deepen. The aggregation formula can be expressed as

$$\boldsymbol{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}} \boldsymbol{e}_i^{(k)} \tag{3.26}$$

$$\boldsymbol{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}} \boldsymbol{e}_u^{(k)} \tag{3.27}$$

where $\boldsymbol{e}_u^{(k)}$ and $\boldsymbol{e}_i^{(k)}$ represents the embedding of user node u and item node i at layer k respectively. This symmetric normalization ensures that the influence of each neighboring node is properly balanced, mitigating the risk of overemphasizing certain nodes. The aggregation process continues until the final layer, where the complete representation of each node is obtained, encapsulating both local and global neighborhood interactions.

After K layers, the final user and item embeddings are obtained by combining it from all layers

$$\boldsymbol{e}_u = \sum_{k=0}^{K} \alpha_k \boldsymbol{e}_u^{(k)} \tag{3.28}$$

$$\boldsymbol{e}_i = \sum_{k=0}^{K} \alpha_k \boldsymbol{e}_i^{(k)} \tag{3.29}$$

where $\alpha_k = \frac{1}{K+1}$ represents the importance of each layer, which can be manually tuned or optimized. Layer combination is important because different layers

capture different semantic information, and the weighted sum approach indirectly accounts for self-connections in GCNs.

The model prediction is the inner product of the final user and item embeddings

$$\hat{y}_{ui} = \boldsymbol{e}_u^\top \boldsymbol{e}_i \tag{3.30}$$

used for ranking in recommendations.

By using LGC, He-LiGCN is able to maintain scalability and computational efficiency, even in large and complex graph structures, while ensuring that the aggregated embeddings accurately reflect the relationships between users and items.

3.4.2. Resolving Over-smoothing Problem with Approximate Personalized Propagation of Neural Predictions

A key challenge in graph neural networks is the over-smoothing problem [69], where node representations become too similar after multiple graph convolution layers (Figure 3.31). To address this issue in He-LiGCN, we adopt APPNP [58] technique, which maintains node distinctiveness while enabling deep architectures.

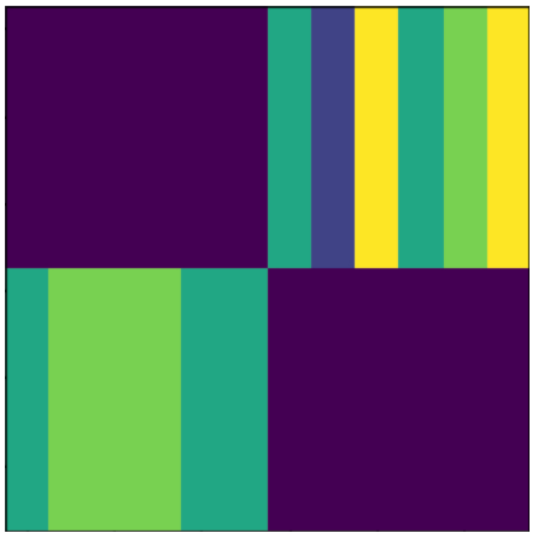


Figure 3.31: Over-smoothing problem.

The APPNP approach modifies the standard graph convolution operation by incorporating a teleportation term that preserves a node's initial features

$$\boldsymbol{E}^{(k+1)} = \beta \boldsymbol{E}^{(0)} + (1-\beta)\left(\boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{A}\boldsymbol{D}^{-\frac{1}{2}}\right)\boldsymbol{E}^{(k)} = \beta \boldsymbol{E}^{(0)} + (1-\beta)\tilde{\boldsymbol{A}}\boldsymbol{E}^{(k)} \tag{3.32}$$

where $\boldsymbol{E}^{(k)}$ is the node representation at layer k, $\beta \in (0,1)$ is the teleportation probability, $\tilde{\boldsymbol{A}}$ is the normalized adjacency matrix, and $\boldsymbol{E}^{(0)}$ represents the initial node features. This formulation allows information to propagate through the graph while maintaining a connection to each node's starting point. The teleportation

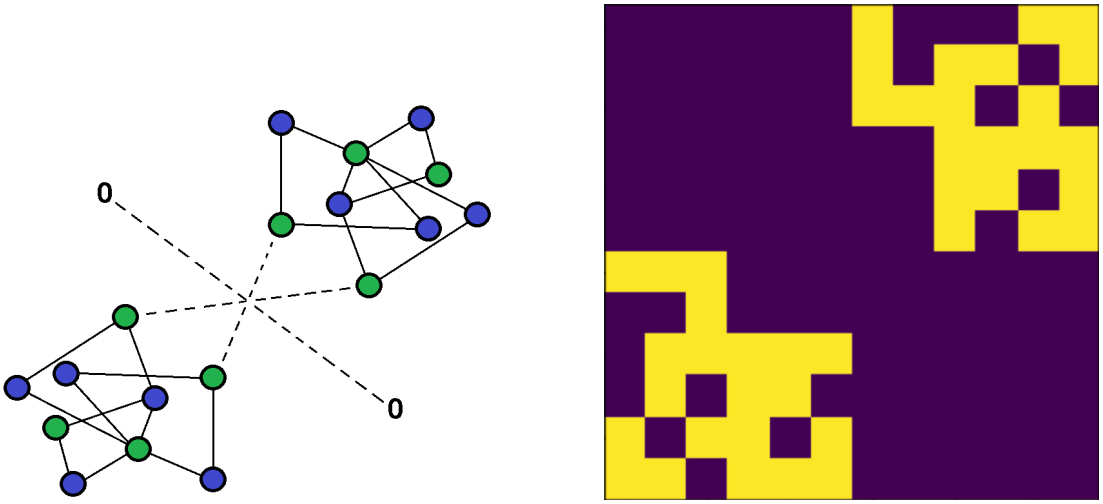


Figure 3.33: Adjacency matrix representation.

term $\beta \boldsymbol{E}^{(0)}$ acts as a residual connection that mitigates the over-smoothing effect by ensuring nodes retain some of their original distinctive features.

For efficient implementation, we use the following iterative approximation from Equation 3.29 and the final node representations are obtained after K iterations. This approach enables He-LiGCN to effectively utilize deeper architectures without suffering from over-smoothing. The teleportation probability β is treated as a hyperparameter and tuned on the validation set. In our experiments, we found $\beta = 0.1$ provides a good balance between information propagation and feature preservation.

3.4.3. Matrix Representation

Once the neighborhood information has been aggregated across multiple graph layers, the system moves on to represent these interactions in a more structured form. The user-item interactions are modeled as a matrix, denoted by $\boldsymbol{R} \in \mathbb{R}^{M \times N}$, where each entry $\boldsymbol{R}_{ui}$ represents whether user u has interacted with item i (with binary values indicating interaction), M and N is the number of u, i respectively.

This matrix is critical for the final recommendation generation, as it forms the basis for computing predicted scores between users and items. To streamline the calculation, the adjacency matrix $\boldsymbol{A}$ is introduced, which represents the structure of the user-item graph. This matrix takes the form

$$\boldsymbol{A} = \begin{pmatrix} \boldsymbol{0} & \boldsymbol{R} \\ \boldsymbol{R}^\top & \boldsymbol{0} \end{pmatrix} \tag{3.34}$$

Here, we create a symmetric matrix that connects users to items and vice versa. This matrix representation serves as the foundation for the information flow in the graph convolution process, where embeddings are progressively refined as they propagate through the layers of the graph.

The initial embedding matrix $\boldsymbol{E}^{(0)} \in \mathbb{R}^{(M+N) \times T}$ is updated at each layer using the normalized adjacency matrix $\tilde{\boldsymbol{A}}$ and a graph convolution operation, with T is

the embedding size. This step consolidates the information from both users and items, ensuring that the embeddings captured at each layer are reflective of both types of nodes in the graph. The final embedding matrix is defined as

$$\boldsymbol{E} = \beta \boldsymbol{E}^{(0)} + \sum_{k=0}^{K} \beta(1-\beta)^k \tilde{\boldsymbol{A}}^k \boldsymbol{E}^{(0)} + (1-\beta)^K \tilde{\boldsymbol{A}}^K \boldsymbol{E}^{(0)} \tag{3.35}$$

3.4.4. Loss Function

To optimize the He-LiGCN framework, the model employs a Bayesian Personalized Ranking (BPR) [70] loss function, which is specifically designed for recommendation systems that handle implicit feedback data. This loss function aims to improve the ranking of items for each user by ensuring that items the user has interacted with are ranked higher than items they have not interacted with.

The BPR loss function works by maximizing the difference between the predicted scores of positive and negative items for a given user

$$\mathcal{L}_{\text{BPR}} = -\sum_{u=1}^{M} \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\boldsymbol{\Theta}\|^2 \tag{3.36}$$

where M represents the total number of users, $\mathcal{N}_u$ denotes the set of items interacted with by user u, and σ is the sigmoid function. The term $\hat{y}_{ui}$ indicates the predicted score for the interaction between user u and item i, while $\hat{y}_{uj}$ represents the score for a negative sample j, which user u has not interacted with. The summation iterates over all users, their positively interacted items, and their negative samples. The loss function aims to maximize the difference $\hat{y}_{ui} - \hat{y}_{uj}$, ensuring that positively interacted items are ranked higher than those not interacted with.

The regularization term $\lambda \|\boldsymbol{\Theta}\|^2$ of the initial embedding matrix $\boldsymbol{E}^{(0)}$ helps prevent overfitting by penalizing large embeddings, ensuring that the learned representations generalize well to new data. By focusing on ranking accuracy, this loss function ensures that the model is optimized to produce the best possible recommendations for each user, based on the learned embeddings. This combination of efficient aggregation, matrix representation, and a well-tuned loss function enables He-LiGCN to deliver accurate, personalized recommendations that scale effectively with large datasets.

4 Experiments and Results

4.1 Experiments

4.1.1. Dataset Preparation

The experiments presented a comprehensive evaluation of the proposed He-LiGCN framework, highlighting its performance, scalability, and computational efficiency. The widely used MovieLens 1M dataset was chosen for the evaluation, consisting of approximately 1 million interactions between 6,040 users and 3,706 movies. Explicit ratings were transformed into binary interactions to simulate implicit feedback scenarios, aligning with real-world recommendation settings. The dataset was divided into training, validation, and test sets in a ratio of 80:10:10, ensuring a robust evaluation setup.

4.1.2. Training Parameters

The training parameters for He-LiGCN and the baseline models were carefully chosen to ensure a fair comparison and optimal performance during the experiments. The settings were consistent across all models, except where specific configurations were required by their architectures (e.g., coarsening levels in He-LiGCN). These parameters were tuned based on prior experiments and validated for robustness in producing meaningful results on the MovieLens 1M dataset. The following table summarizes the key parameters used during training

Table 4.1: Training Parameters for He-LiGCN, NGCF, and LightGCN

Parameter	**He-LiGCN**	**NGCF**	**LightGCN**
Embedding Size	64	64	64
Learning Rate	0.001	0.001	0.001
Batch Size	1024	1024	1024
Number of Layers	3	3	3
Coarsening Levels	3	N/A	N/A
Regularization Coefficient (λ)	10^{-2}	10^{-2}	10^{-2}
Optimizer	Adam	Adam	Adam

This table provides a concise summary of the training configurations, highlighting the consistency in embedding size, learning rate, and optimizer choice across models. The coarsening levels parameter, specific to He-LiGCN, illustrates its use of multi-level coarsening to reduce graph complexity. The inclusion of regularization parameters ensures robust performance while mitigating overfitting, and the batch size was kept constant for computational fairness.

By structuring this information in a table, readers can quickly grasp the training conditions, facilitating reproducibility and providing a clear link to the experimental results discussed earlier.

4.1.3. Evaluation Metrics

To assess the recommendation quality, we employed two standard evaluation metrics: Recall [71] and NDCG (Normalized Discounted Cumulative Gain) [72]. Recall@k measures the proportion of relevant items retrieved within the top k recommendations, reflecting the model's retrieval performance. NDCG@k quantifies the ranking quality by prioritizing the correct placement of relevant items in the top positions, making it critical for scenarios where user satisfaction depends heavily on highly ranked items. These metrics were selected due to their relevance in evaluating both retrieval and ranking performance.

Recall@10: This metric evaluates the proportion of relevant items retrieved within the top 10 recommendations. It is defined as

$$\text{Recall@10} = \frac{\textbf{Number of relevant items retrieved in top 10}}{\textbf{Total number of relevant items}}$$

It reflects the model's ability to include relevant items in the top-ranked set, making it crucial for assessing retrieval effectiveness.

NDCG@10 (Normalized Discounted Cumulative Gain): This metric evaluates the ranking quality of the top 10 recommendations by penalizing the incorrect placement of relevant items. It is defined as

$$\text{NDCG@10} = \frac{\text{DCG@10}}{\text{IDCG@10}}$$

where DCG@10 is the discounted cumulative gain for the top 10 recommendations and IDCG@10 is the ideal discounted cumulative gain, representing the best possible ranking for the given data.

DCG quantifies the ranking quality of a recommendation list by summing the relevance scores of items, adjusted by a logarithmic discount factor based on their positions in the ranked list. The formula for DCG is

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}$$

Here, rel_i denotes the relevance score of the item at position i, and $\log_2(i+1)$ is the discount factor that decreases the weight of items as their rank increases. This discount reflects the intuitive idea that higher-ranked items are more important for user satisfaction. For example, an item in the first position contributes fully to the score, while items ranked lower contribute less due to the logarithmic scaling.

IDCG is calculated using the same formula as DCG but is applied to the ideal ranking of the items. This ideal ranking orders all relevant items at the top, ensuring the highest possible DCG score for the dataset. IDCG serves as the normalization factor in the NDCG formula, ensuring that the metric is bounded between 0 and 1. This allows for a fair comparison of ranking performance across datasets with varying numbers of relevant items. The formula for IDCG is

$$\text{IDCG@k} = \sum_{i=1}^{k} \frac{\text{rel}_i^{\text{ideal}}}{\log_2(i+1)}$$

where $\text{rel}_i^{\text{ideal}}$ represents the relevance score of the item at position i in the ideal ranking.

4.2 Results

We compared He-LiGCN with two baseline models: NGCF, a model that utilizes non-linear message passing for capturing high-order connectivity, and LightGCN, known for its simplified and computationally efficient graph convolution operations. All models were trained under the same conditions, with consistent hyperparameters, to ensure a fair and robust comparison.

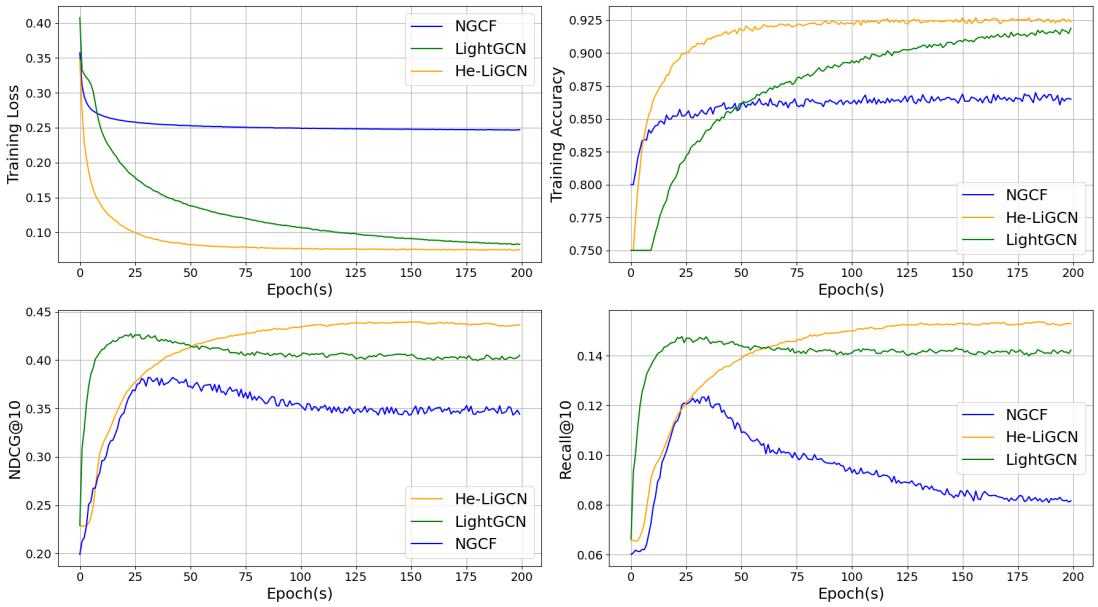


Figure 4.2: The training performance of NGCF, LightGCN, and He-LiGCN models over 200 epochs.

4.2.1. Comparative Analysis of Recommendation Models

The performance of He-LiGCN was comprehensively assessed in terms of trainiing efficiency, scalability across different model depths, and the impact of individual components on its overall accuracy. Figure 4.2 illustrates the training performance of NGCF, LightGCN, and He-LiGCN over 200 epochs. He-LiGCN exhibits the fastest convergence, stabilizing at a training loss of 0.06 within 25 epochs, whereas LightGCN and NGCF require more epochs and converge at higher loss values of 0.12 and 0.24, respectively. This rapid convergence highlights the efficiency of the multi-level graph coarsening and the attention-weighted aggregation mechanisms employed by He-LiGCN.

In terms of accuracy, He-LiGCN consistently achieves superior Recall@10 and NDCG@10 compared to the baselines. Specifically, He-LiGCN outperforms LightGCN and NGCF by 6.5% and 27.5%, respectively, in Recall@10, and by 8.9% and 20.4% in NDCG@10. These results underscore the effectiveness of the proposed

framework in learning accurate user-item representations and delivering high-quality recommendations.

4.2.2. Scalability and Efficiency Across Graph Layers

Table 4.3: Performance comparison at different layer depths.

# Layers	Model	NDCG@10	Recall@10
1	NGCF	0.379	0.120
	LightGCN	0.423	0.140
	He-LiGCN	0.444	0.155
2	NGCF	0.382	0.123
	LightGCN	0.420	0.147
	He-LiGCN	0.446	0.156
3	NGCF	0.377	0.126
	LightGCN	0.429	0.148
	He-LiGCN	**0.448**	**0.157**
4	NGCF	0.370	0.124
	LightGCN	0.428	0.147
	He-LiGCN	0.447	0.150

The scalability of He-LiGCN was evaluated by varying the number of graph convolutional layers. As shown in Table 4.3, He-LiGCN achieves optimal performance at a depth of three layers, with Recall@10 and NDCG@10 reaching 0.157 and 0.448, respectively. Beyond three layers, both NGCF and LightGCN experience performance degradation due to over-smoothing, a phenomenon where node representations become indistinguishable in deep graph convolution networks. In contrast, He-LiGCN maintains its performance by effectively mitigating over-smoothing through its multi-level coarsening and attention-weighted aggregation, preserving the discriminative power of node embeddings even in deeper architectures.

4.2.3. Ablation and Effectiveness Studies

Table 4.4: Ablation study results for He-LiGCN, showing Recall@10 and NDCG@10 performance with removed components (1: graph coarsening; 2: GATNE and HGCN; 3: LGC).

Model Variant	Recall@10	NDCG@10
He-LiGCN-[1, 2]	0.148	0.435
He-LiGCN-[1, 3]	0.141	0.420
He-LiGCN-[2, 3]	0.152	0.442
Complete He-LiGCN	**0.157**	**0.448**

To further investigate the contribution of individual components, we conducted an ablation study by systematically removing key elements of the He-LiGCN framework. As summarized in Table 4.4, the removal of the graph coarsening mechanism led to a significant decrease in Recall@10 and NDCG@10, highlighting its critical role in simplifying large-scale graphs while preserving essential structural

information. Similarly, excluding the GATNE and HGCN components resulted in the most substantial performance drop, emphasizing their importance in capturing the heterogeneous relationships within the graph. Finally, the absence of LGC reduced the model's efficiency in propagating information across layers, further confirming its necessity for achieving optimal performance.

Table 4.5: Average computational cost per epoch for He-LiGCN, NGCF, and LightGCN.

Model	Training time (seconds)
He-LiGCN	10.2
NGCF	12.5
LightGCN	**9.8**

The results in Table 4.5 indicate that He-LiGCN achieves a favorable balance between computational efficiency and model complexity. Although LightGCN exhibits the lowest training time per epoch at 9.8 seconds, He-LiGCN requires only a slightly higher average time of 10.2 seconds, representing a mere 4.1% increase. Despite the slight increase in computational cost compared to LightGCN, He-LiGCN provides significantly better performance in terms of both Recall@10 and NDCG@10. This efficiency-accuracy trade-off underscores the effectiveness of He-LiGCN's design in maintaining computational efficiency while delivering superior recommendation accuracy. By optimizing the balance between model complexity and computational cost, He-LiGCN proves to be a practical and scalable solution for real-world recommendation systems, where both accuracy and efficiency are critical.

Overall, the experimental results validate the robustness and efficiency of He-LiGCN. By integrating multi-level graph coarsening, heterogeneous embeddings, and lightweight convolution operations, He-LiGCN demonstrates superior accuracy, scalability, and computational efficiency compared to differents models. These findings highlight the potential of He-LiGCN as a practical and scalable solution for personalized recommendation systems in large-scale, heterogeneous environments.

5 Conclusion

5.1 Key Contributions

With all the challenges that we have been addressed, He-LiGCN had been proposed that integrated multi-level graph coarsening, heterogeneous embeddings, and lightweight convolution. This design allows He-LiGCN to efficiently model complex user-item interactions while maintaining a lightweight and scalable architecture. The multi-level graph coarsening reduces computational costs by simplifying the graph structure without sacrificing essential relationships. Meanwhile, heterogeneous embeddings capture diverse user-item interactions, and LGC ensures efficient information aggregation across multiple layers.

The experimental evaluation on the MovieLens 1M dataset highlights the significant contributions of He-LiGCN. The framework outperformed these two models, achieving a 6.5% improvement in Recall@10 and an 8.9% increase in NDCG@10 compared to LightGCN, while maintaining a competitive average training time of only 10.2 seconds per epoch. This demonstrates that He-LiGCN not only provides more accurate recommendations but also achieves faster convergence and lower computational overhead compared to existing models. Furthermore, the ablation studies confirmed the critical role of each component in the overall performance of He-LiGCN, with the integration of graph coarsening and heterogeneous embeddings being particularly impactful in preserving the discriminative power of node representations across multiple layers.

By addressing both the accuracy and scalability dimensions, He-LiGCN represents a significant advancement in graph-based recommendation systems. Its design principles and methodological innovations pave the way for more efficient and scalable personalized recommendation systems, capable of handling complex, large-scale datasets across a wide range of application domains.

5.2 Limitations

Despite the promising results, He-LiGCN has certain limitations that warrant further investigation. One of the primary challenges lies in the multi-level graph coarsening technique. While it effectively reduces computational complexity by simplifying the graph structure, there is a risk of minor information loss, particularly in densely connected regions. Although the subsequent refinement stages using heterogeneous embeddings help mitigate this loss, it may still impact recommendation accuracy in scenarios where preserving fine-grained relationships is critical. Another limitation is the assumption of a static graph structure throughout the training and recommendation processes. This static assumption limits the framework's adaptability to dynamic environments, where user preferences and item characteristics evolve over time, such as in real-time recommendation systems or streaming data contexts. As a result, He-LiGCN may not fully capture

temporal patterns or rapidly changing user behaviors. Furthermore, the current implementation of He-LiGCN relies on predefined hyperparameters and fixed coarsening levels, which may reduce its flexibility when applied to datasets with varying structural properties. Although these parameters were carefully tuned for optimal performance on the MovieLens 1M dataset, their generalizability to other datasets with different characteristics remains a potential area for improvement.

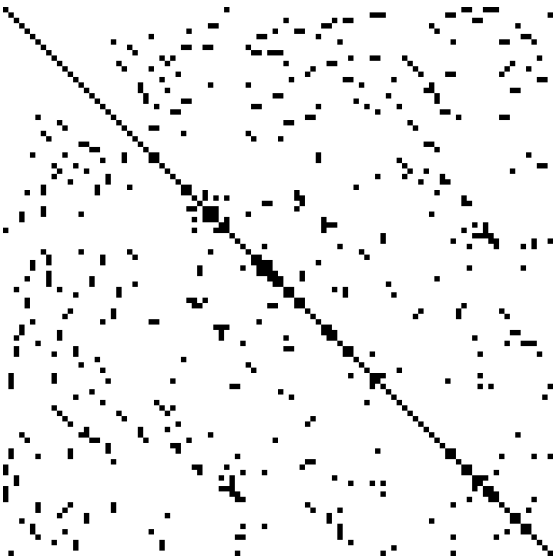


Figure 5.1: Sparse matrix representation.

5.3 Future Work

To further enhance He-LiGCN, several key directions are proposed. First, the development of adaptive graph coarsening techniques could dynamically adjust the coarsening level based on the graph's density, preserving critical information while maintaining computational efficiency. Second, extending He-LiGCN to support dynamic graph modeling would enable real-time recommendations by incorporating temporal information and continuously updating embeddings as user-item interactions evolve. Third, integrating self-supervised learning methods could reduce the reliance on labeled data, improving performance in domains with sparse interactions. This enhancement would make He-LiGCN more adaptable across various datasets. Finally, future work could focus on increasing the interpretability of the model through attention mechanisms, providing insights into the importance of nodes and edges in recommendation decisions. Expanding evaluations across diverse datasets and domains such as e-commerce, education, and healthcare would further validate the scalability and applicability of He-LiGCN in real-world scenarios.

References

Back-references to the pages where the publication was cited are given by •.

[1] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, **2019**.

1

[2] Jie Lu, Dianshuang Wu, Mingsong Mao, Wei Wang, and Guangquan Zhang. Recommender system application developments: A survey. *Decision Support Systems*, **2015**.

1

[3] McKinsey and Company. The value of getting personalization right—or wrong—is multiplying. tech. rep. McKinsey and Company, **2021**.

1

[4] Carlos A Gomez-Uribe and Neil Hunt. The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems*, **2015**.

1

[5] Gediminas Adomavicius and Alexander Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, **2005**.

1

[6] Eli Pariser. The filter bubble: What the Internet is hiding from you. Penguin UK, **2011**.

1

[7] Michael J Pazzani and Daniel Billsus. Content-based recommendation systems. *The Adaptive Web*, **2007**.

1

[8] Andrew I Schein, Alexandrin Popescul, Lyle H Ungar, and David M Pennock. Methods and metrics for cold-start recommendations. *Proceedings of the 25th Annual International ACM SIGIR Conference*, **2002**.

1

[9] Xiaoyuan Su and Taghi M Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in Artificial Intelligence*, **2009**.

1

[10] Yehuda Koren and Robert Bell. Advances in collaborative filtering. *Recommender Systems Handbook*, **2015**.

1

[11] Xiao Huang, Jingyuan Xu, Qiang Li, Yongfeng Liu, et al. Meta-learning on

heterogeneous information networks for cold-start recommendation. *Proceedings of the 26th ACM SIGKDD International Conference*, **2020**.

2

[12] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, **2020**.

2

[13] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. IN: *Proceedings of the 24th ACM SIGKDD International Conference*. **2018**.

2

[14] Shiqing Cheng, Ziyi Liu, Jiliang Li, and Yu-Gang Jiang. HGCF: A hierarchical graph convolution framework for fair recommendation. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, **2020**.

2, 6

[15] Yanbang Cen, Xu Zou, Jianwei Zhang, Hongxia Yang, Jie Zhou, and Jie Tang. Representation learning for attributed multiplex heterogeneous network. IN: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. **2019**.
DOI: 10.1145/3292500.3330965

2, 6

[16] Shoujin Wang, Liang Hu, Yan Wang, Xiangnan He, Longbing Cao, et al. Graph learning for recommender systems: A survey. *ACM Computing Surveys*, **2022**.

3

[17] Andrew Zhang, Xianghao Yao, and Jian Sun. Web-scale recommender systems at Tencent. *Proceedings of the 26th ACM SIGKDD International Conference*, **2020**.

3

[18] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. *Proceedings of the 24th ACM SIGKDD International Conference*, **2018**.

3

[19] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. Graph neural networks for social recommendation. *The World Wide Web Conference*, **2019**.

3

[20] John Adrian Bondy and Uppaluri Siva Ramachandra Murty. Graph theory with applications. Macmillan London, **1976**.

3

[21] Douglas Brent West. Introduction to graph theory. Prentice Hall, **2001**. 3

[22] Rex Ying, Ruining He, Kaifeng Chen, Chaklam Eksombatchai, William L Hamilton, and Jure Leskovec. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. IN: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM. **2018**.
DOI: 10.1145/3219819.3219890 3

[23] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. *Proceedings of the 20th ACM SIGKDD International Conference*, **2014**. 3, 5

[24] Douglas B. West. Introduction to Graph Theory. 2nd. Prentice Hall, **2001**. 4

[25] Maksim Kitsak, Lazaros K Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H Eugene Stanley, and Hernán A Makse. Identification of influential spreaders in complex networks. *Nature Physics*, **2010**.
DOI: 10.1038/nphys1746 4

[26] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S Yu. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, **2016**. 4

[27] Zhao Yang, René Algesheimer, and Claudio J Tessone. Community detection in networks: An in-depth benchmarking study. *Scientific Reports*, **2013**. 4

[28] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. Heterogeneous graph attention network. *The World Wide Web Conference*, **2019**. 4, 5

[29] Yizhou Sun and Jiawei Han. Mining heterogeneous information networks: a structural analysis approach. *ACM SIGKDD Explorations Newsletter*, **2013**. 4

[30] Carl Yang, Yuxin Xiao, Yuan Zhang, Yizhou Sun, and Jiawei Han. Heterogeneous network representation learning: Survey, benchmark, evaluation, and beyond. *IEEE Transactions on Knowledge and Data Engineering*, **2020**. 4

[31] William L Hamilton, Rex Ying, and Jure Leskovec. Representation learning on graphs: Methods and applications. *IEEE Data Engineering Bulletin*, **2017**. 4

[32] Hongyun Cai, Vincent W Zheng, and Kevin Chen-Chuan Chang. A com-

prehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering*, **2018**.

4

[33] Zhiwei Liu, Chen Chen, Jun Zhou, Jizhou Li, et al. Graph reduction: A survey and lessons learned. *ACM Computing Surveys*, **2022**.

4

[34] Jure Leskovec and Christos Faloutsos. Sampling from large graphs. *Proceedings of the 12th ACM SIGKDD International Conference*, **2006**.

4

[35] Xiawu Zheng, Gao Huang, Shilei Jiang, and Wei Li. Graph pruning for model compression. *Pattern Recognition Letters*, **2020**.

4

[36] Daokun Zhang, Jie Yin, Xingquan Zhu, and Chengqi Zhang. Network representation learning: A survey. *IEEE Transactions on Big Data*, **2018**.

5

[37] Jiongqian Liu, Sandya Kumar, and Jingbo Wang. MILE: A Multi-Level Framework for Scalable Graph Embedding. *ACM Transactions on Knowledge Discovery from Data*, **2021**.

5

[38] Fenyu Hong, Yaser Huang, and Wei Xie. Attention-based graph neural network for semi-supervised learning. *arXiv preprint arXiv:2003.13198*, **2020**.

5

[39] Xinyu Fu, Jiani Zhang, Zhen Meng, and Irwin King. MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding. *The Web Conference*, **2020**.

5

[40] Peng Cui, Xiao Wang, Jian Pei, and Wenwu Zhu. A survey on network embedding. *IEEE Transactions on Knowledge and Data Engineering*, **2019**.

5

[41] Palash Goyal and Emilio Ferrara. Graph embedding techniques, applications, and performance: A survey. *Knowledge-Based Systems*, **2018**.

5

[42] Yixin Liu, Shirui Jian, Shirui Pan, Feng Zhou, and Xue Gao. Representation learning for networks: A survey. *ACM Computing Surveys*, **2021**.

5

[43] Yukuo Cen, Xu Zou, Jianxin Zhang, Hongxia Yang, Jingren Zhou, and Jie Tang. Representation learning for attributed multiplex heterogeneous network. *Proceedings of the 25th ACM SIGKDD International Conference*, **2019**.

5

[44] Haochen Chen, Bryan Perozzi, Yifan Hu, and Steven Skiena. Tutorial: Net-

work embedding. *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, **2018**.

6

[45] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD International Conference*, **2016**.

6

[46] HongYun Cai, Vincent W Zheng, and Kevin Chen-Chuan Chang. A comprehensive survey of graph learning: Progress and opportunities. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **2020**.

6

[47] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, **2017**.

6

[48] Zhitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, Will Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. *Advances in Neural Information Processing Systems*, **2018**.

6

[49] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Peng Cui, Philip S Yu, and Yanfang Ye. A survey on heterogeneous graph embedding: Methods, techniques, applications and sources. *Information Processing and Management*, **2020**.

6

[50] Huichen Hong, Hao Guo, Yucheng Lin, Xiaochun Yang, Zhu Li, and Jingrong Ye. Heterogeneous graph attention networks for semi-supervised short text classification. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, **2019**.

6

[51] Yingxue Zhang, Yawei Xiong, Xiaosong Kong, Shaofeng Li, Jianliang Mi, and Yongyuan Zhu. Heterogeneous graph convolutional networks for text classification. *Proceedings of the AAAI Conference on Artificial Intelligence*, **2019**.

6

[52] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. Heterogeneous attention networks for recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, **2019**.

6

[53] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. *Proceedings of The Web Conference 2020*, **2020**.

6

[54] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and

Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, **2009**.
DOI: 10.1109/TNN.2008.2005605 7

[55] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xiao Hu, and Tat-Seng Chua. Neural collaborative filtering. IN: *Proceedings of the 26th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee. **2017**.
DOI: 10.1145/3038912.3052569 7

[56] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. Neural Graph Collaborative Filtering. IN: *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, **2019**.
DOI: 10.1145/3331184.3331267 7

[57] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. LightGCN: Simplifying and powering graph convolution network for recommendation. IN: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. **2020**.
DOI: 10.1145/3397271.3401063 7

[58] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized PageRank. IN: *International Conference on Learning Representations (ICLR)*. **2019**.
7, 21

[59] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, **2018**.
7

[60] Hanqing Zhang and Ian Goodfellow. Revisiting graph convolutional networks: Neighborhood aggregation and network sampling. *arXiv preprint arXiv:2002.01935*, **2020**.
7

[61] Deli Chen, Yankai Lin, Wei Wei, Peng Li, Jie Zhou, and Xu Sun. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. *Proceedings of the AAAI Conference on Artificial Intelligence*, **2020**.
8

[62] Patrick Sathre, Anurag Gondhalekar, and Wu-Chun Feng. Edge-Connected Jaccard Similarity for Graph Link Prediction on FPGA. IN: *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, **2022**.
DOI: 10.1109/HPEC55875.2022.9904847 10

[63] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. A Survey on Locality Sensitive Hashing Algorithms and their Applications. **2021**.

ArXiv: 2102.08942 [cs.DB] URL: https://arxiv.org/abs/2102.08942 10

[64] Reuven Y. Rubinstein and Dirk P. Kroese. The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning. Springer, **2004**.
DOI: 10.1007/978-1-4757-4321-0 15

[65] Sebastian Ruder. An overview of gradient descent optimization algorithms. **2017**.
ArXiv: 1609.04747 [cs.LG] URL: https://arxiv.org/abs/1609.04747 15

[66] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. Rectifier Nonlinearities Improve Neural Network Acoustic Models. IN: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. **2013**.
URL:
https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf
18

[67] Erich L Lehmann and George Casella. Theory of Point Estimation. 2nd ed. Springer Science and Business Media, **2006**.
19

[68] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*, **2014**.
ArXiv: 1412.6980 [cs.LG] URL: https://arxiv.org/pdf/1412.6980 19

[69] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. A Survey on Oversmoothing in Graph Neural Networks. **2023**.
ArXiv: 2303.10993 [cs.LG] URL: https://arxiv.org/abs/2303.10993 21

[70] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian Personalized Ranking from Implicit Feedback. **2012**.
ArXiv: 1205.2618 [cs.IR] URL: https://arxiv.org/abs/1205.2618 23

[71] David M. W. Powers. Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*, **2011**. Archived from the original (PDF) on 2019-11-14.
URL: https://www.researchgate.net/publication/281327253 25

[72] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, Wei Chen, and Tie-Yan Liu. A Theoretical Analysis of Normalized Discounted Cumulative Gain (NDCG) Ranking Measures. IN: *Proceedings of the 26th Annual Conference on Learning Theory (COLT 2013)*. JMLR Workshop and Conference Proceedings, **2013**.
25